

AI-DJ: Spotify Hit Predictor

Presented by : Antoine Paulis
Charlotte Michon
Mohamed-Khalil Ankri

Presentation Outline

- **Project Definition & Scope**
- **MLOps Automation**
- **Data Strategy: EDA**
- **Machine learning models**

Project Definition & Scope

- Project proposal : Tool to **predict** if a song will be a “**hit**” or a “**flop**”, by training on a Spotify dataset of different genres of music.
- Project value proposition :
 - **Marketing goal** : choose which titles could have the biggest impact to resonate with the audience of an ads
 - **Radio diffusion** : identify potential hits that could match current trends
 - **DJ performance** : choose best combinations of songs to animate parties
 - **Independent artists** : test different versions or mix of a track, to see which one could corresponds the best to actual “market” standards

retaken from project card we submitted

Project Definition & Scope

- Scope :
 - Data: Spotify audio features
 - Model: Binary classification (Hit vs. Flop) based on popularity thresholds
 - System: A functional API and a web dashboard
 - (Out of scope: Raw audio files (.mp3) or analyzing song lyrics (NLP))

MLOps Automation

- github repository :

- docs: **.md files** with written explanations, and **.pdf figures**
- infra: notebook for **Google Cloud Storage** implementation - for the project, only bucket to use the dataset
- src: **.py** code files and **notebook** - models in training folder, other files are empty
- tests: python code for pytest

- git flow strategy :

- merge to integrate changes and features to develop branch
- no merge conflicts occurred (only some local problems on synchronization for one of us)

```
info9023-ai-dj/
├── .github/workflows/      # CI/CD/CT (GitHub Actions)
│   ├── ci.yaml            # Runs Python & Next.js tests on PRs
│   ├── deploy-api.yaml    # Deploys FastAPI to Cloud Run
│   └── deploy-web.yaml     # Deploys Next.js to Cloud Run/Vercel
├── data/                  # Data folder (tracked by DVC)
│   ├── raw/               # Original Kaggle CSV (DVC pointer)
│   └── processed/         # Transformed data for training
├── docs/                  # Course Documentation
│   ├── DEPLOYMENT.md      # Steps to build Docker images
│   ├── MONITORING.md      # Drift detection logic (Summer/Winter)
│   └── API_SPEC.md        # JSON request/response examples
├── src/                   # BACKEND (FastAPI)
│   ├── api/               # API Routes & Spotify logic
│   │   ├── main.py        # Pydantic models (Data Validation)
│   │   ├── schemas.py     # Loads the .pkl or MLflow model
│   │   └── model_loader.py # spotipy client wrapper
│   ├── ui/                # FRONTEND (Streamlit)
│   │   ├── components/    # Radar charts, Hit gauges
│   │   ├── Templates/
│   │   └── app.py          # Main Streamlit application
│   ├── training/          # ML PIPELINE
│   │   ├── train.py        # Training script (XGBoost/LightGBM)
│   │   ├── evaluate.py     # Logs metrics to MLflow
│   │   └── monitoring/     # DRIFT DETECTION
│   │       └── drift_service.py # Checks for "Musical Seasonal Drift"
├── tests/                 # Quality Assurance
│   └── test_api.py         # FastAPI endpoint tests
├── Dockerfile.api         # Container for the Backend
├── Dockerfile.ui          # Container for the Streamlit
├── .pre-commit-config.yaml # Hooks for ruff, trailing-whitespace, etc.
├── ruff.toml              # Linting & Formatting rules
├── dvc.yaml               # DVC Pipeline (Data -> Train -> Eval)
├── requirements.txt        # Python dependencies
├── package.json            # Next.js dependencies
├── README.md              # Project landing page
├── .gitignore              # Git ignore rules
└── utils                  # Additional config files
```

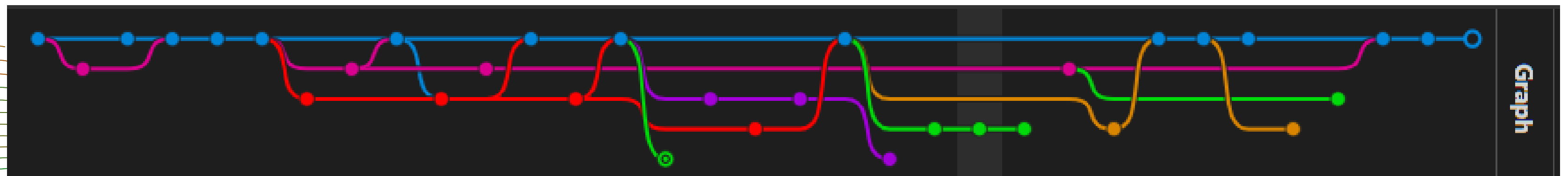
illustration of the initial repository structure: small modifications occurred during work, but main idea for structure is still the same

MLOps Automation

- points of amelioration:

- merge earlier branches to avoid “crossed-nodes”:
 - reduce live time of features branches
 - merge them as soon as one task is finished (or at least sketched)
- delete remote/local branches earlier, and feature branches implemented only for the lab session: clean more frequently branches since it is not totally clear what changes were made exactly
- maintain clearer history:
 - consider less merge commits
 - use rebase to integrate more often our develop fonctionnalites (that we discussed together to assess utility)

```
○ develop origin re-run with good outputs
ruff style : reorder import and path from gcs
lab4-cha Merge branch 'feature/eda' into develop
feature/eda origin sort import for training files, downloaded data from GC
origin/ml2-system-skeleton skeleton code init for next milestone 2
modified path from GCS
edits on Readme
Merge pull request #4 from khalilAnkri/structure-adjustements
origin/structure-adjustements final touches on the repo structure
Add ML part in explanation + code changes
assignment3 origin typo name python file
trying to find good path
train from Assistant
lab3 origin dockerfile cha
Merge branch 'gcp-Setup' into develop
docker file and flask app added
origin/gcp-Setup gcp setup fix
updated flask requirements
stash@{0} WIP on main: f850e7a Merge branch 'gcp-Setup' into develop
main Merge branch 'gcp-Setup' into develop
gcp setup config
Merge pull request #3 from khalilAnkri/gcp-Setup
EDA + 3 ML models
Merge branch 'develop' of https://github.com/khalilAnkri/AI-Dj into develop
Merge pull request #2 from khalilAnkri/feature/eda
Add EDA notebook
origin/HEAD origin/main final touches on the repo structure
adding test files
added ruff and pre-commit configurations
Merge branch 'main' of https://github.com/khalilAnkri/AI-Dj
added Documentation files
Remove citations from readme.md
init commit
```



generated git graph in VSCode

MLOps Automation

CI/CD Pipeline: Automated testing (future work), Docker builds (next milestone assignment), and deployments via GitHub Actions (workflow for next milestones).

Cloud Data Management: Utilizing a Google Cloud Storage (GCS) bucket to securely store, version, and manage our large Spotify dataset and model artifacts outside of our codebase.

Configurations: According to project guidelines...

- Code quality check: **Ruff** for Python code; fast and all-in-one (both linting and formatting)
- Pytest: folder tests, mainly used for labs
- Pre-commit hooks: only **.yaml** file for setup



GitHub Actions



Google Cloud

Data Strategy : EDA

- **Dataset** : **21** variables and **114,000** observations/tracks (125 genres).
 - kaggle
- **Metadata & Identifiers** :
 - track_id, artists, album_name, track_name and track_genre.
- **Audio features** :
 - **Rythm and intensity** : danceability, energy, tempo and loudness.
 - **Mood** : valence and mode.
 - **Sound composition** : acousticness, instrumentalness, speechiness, liveness and explicit.
 - **Structure** : key, time_signature and duration_ms.

Data Strategy : EDA

- Audio features :
 - **Different scales :** 0-1, integer, ms, True/False, BPM and dB.

Feature	Type	Range / Format	Description
danceability	Float	0.0 - 1.0	Suitability for dancing (tempo, rhythm stability).
energy	Float	0.0 - 1.0	Perceptual measure of intensity and activity.
valence	Float	0.0 - 1.0	Musical positiveness (Happy/Cheerful vs. Sad/Angry).
tempo	Float	BPM	The estimated speed of the track.
loudness	Float	dB	Overall loudness of the track.
speechiness	Float	0.0 - 1.0	Presence of spoken words (0.66+ = likely spoken word).
acousticness	Float	0.0 - 1.0	Confidence measure of whether the track is acoustic.
instrumentalness	Float	0.0 - 1.0	Likelihood the track contains no vocal content.
liveness	Float	0.0 - 1.0	Presence of an audience in the recording.
key	Integer	-1 to 11	Pitch class notation (0 = C, 1 = C#, etc.).
mode	Integer	0 or 1	Modality (1 = Major, 0 = Minor).
duration_ms	Integer	ms	Length of the track in milliseconds.
time_signature	Integer	3 to 7	Estimated beats per bar.
explicit	Boolean	True/False	Whether the track contains explicit lyrics.

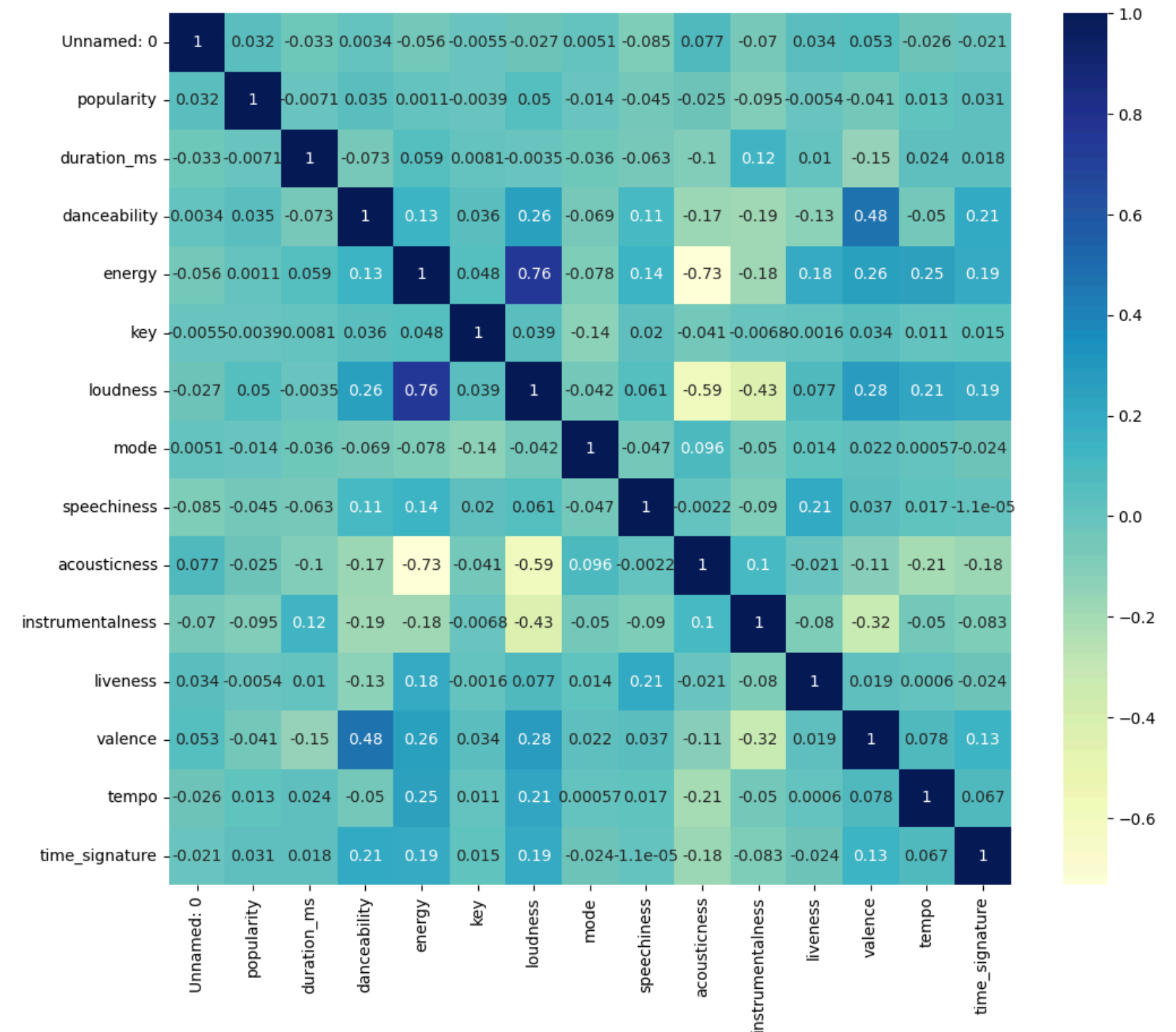
Data Strategy : EDA

- Target variable :
 - Initial target variable : Popularity (0-100).
 - 2 factors : total number of streams and how recent those streams are.
 - New target variable : **Hit** (0-1).
 - **Threshold** at 65 : 8.31% hit (9477).
- Missing values :
 - **1 observation** with **3 missing variables** : artists, album_name and track_name.

Data Strategy : EDA

- Correlation matrix :

- Highest positive correlation: **0.76** between loudness and energy.
- Highest negative correlation: - **0.73** between energy and acousticness.
- Popularity **does not show strong correlation** => non-linear model.



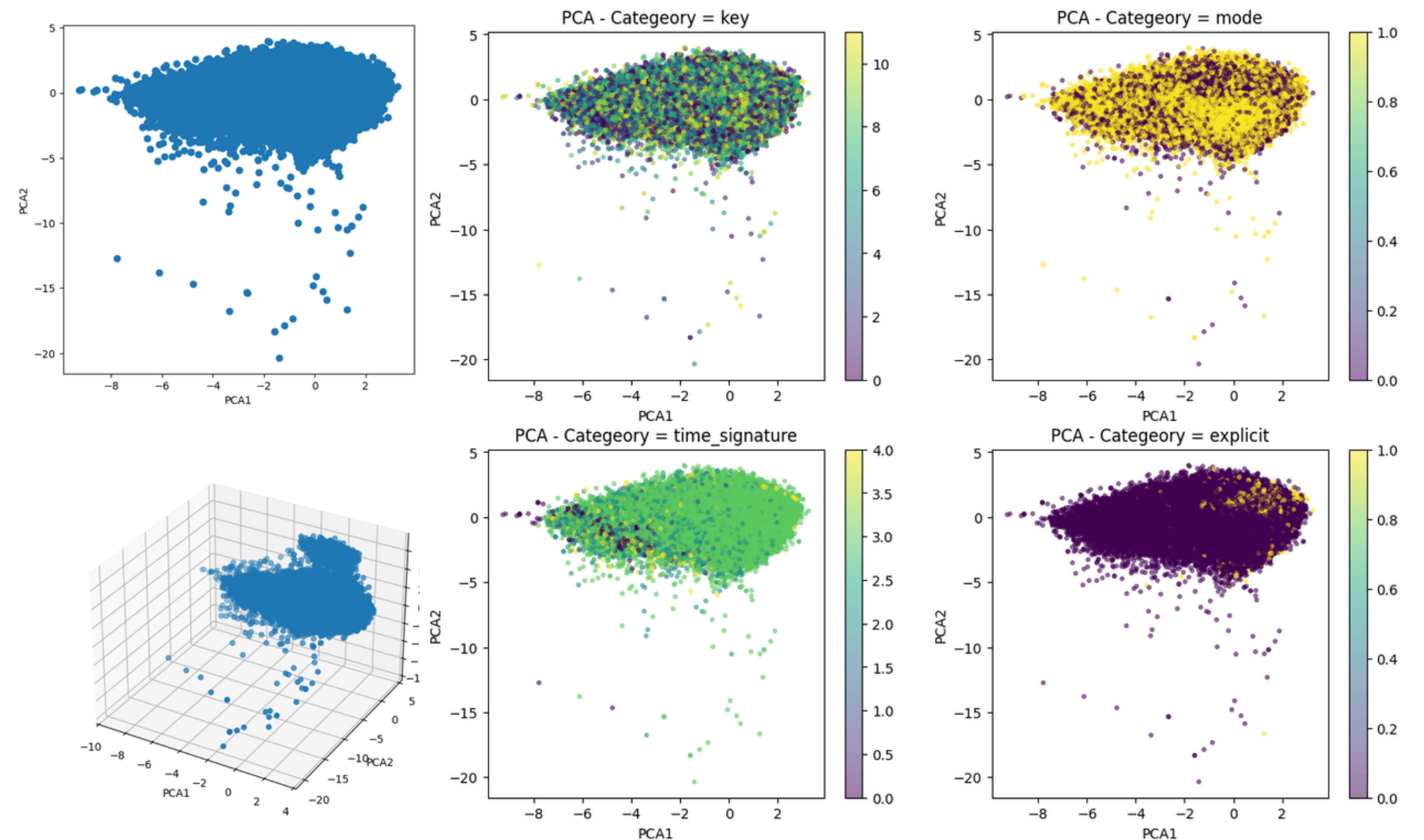
Data Strategy : EDA

- **PCA :**

- Explained variances:
 - PC1 : 28%, PC2 : 15% and PC3 : 12%
- No clear cluster.
- No clear distinction by the categorical variables.

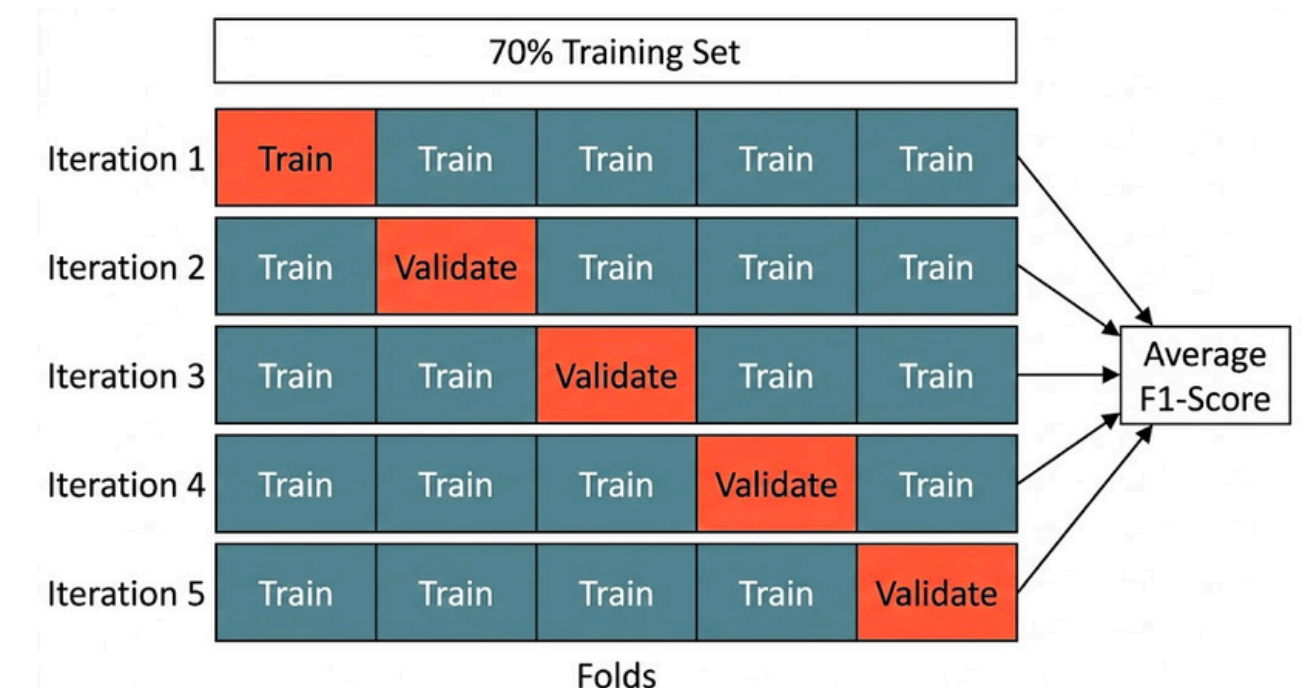
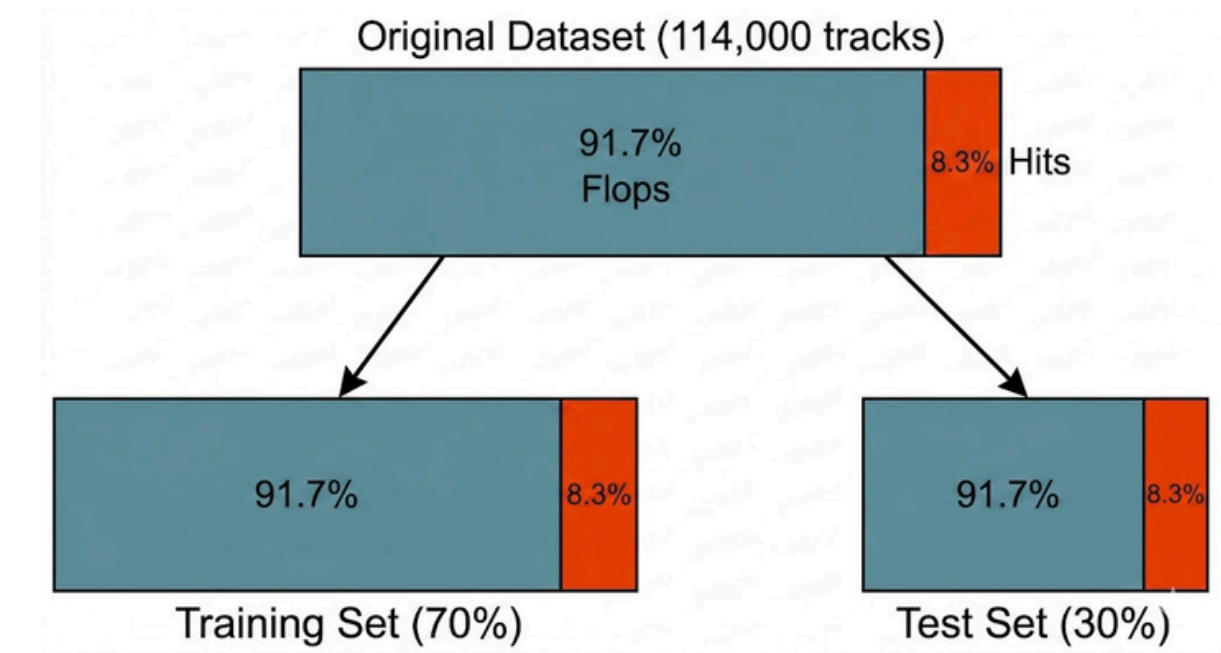
- **Implication for ML modelling :**

- Strong imbalance (8.31 % of hit).
- No clear cluster.
- No linear trend.



ML Models

- **The Goal:** Classify Spotify tracks as "Hits" (Popularity over 65) vs. "Flops".
- **The Challenge:** Extreme Class Imbalance (Only 8.31% Hits).
- **Experiment Setup:**
 - **Data Split:** 70/30 Stratified Split .
 - **Validation:** Stratified K-Fold Cross-Validation (with shuffle).
 - **Algorithms Tested:** KNN, Random Forest, Gradient Boosting.
- **Decision Metric:** F1-Score , Prioritized over Accuracy to handle the class imbalance.



ML Models

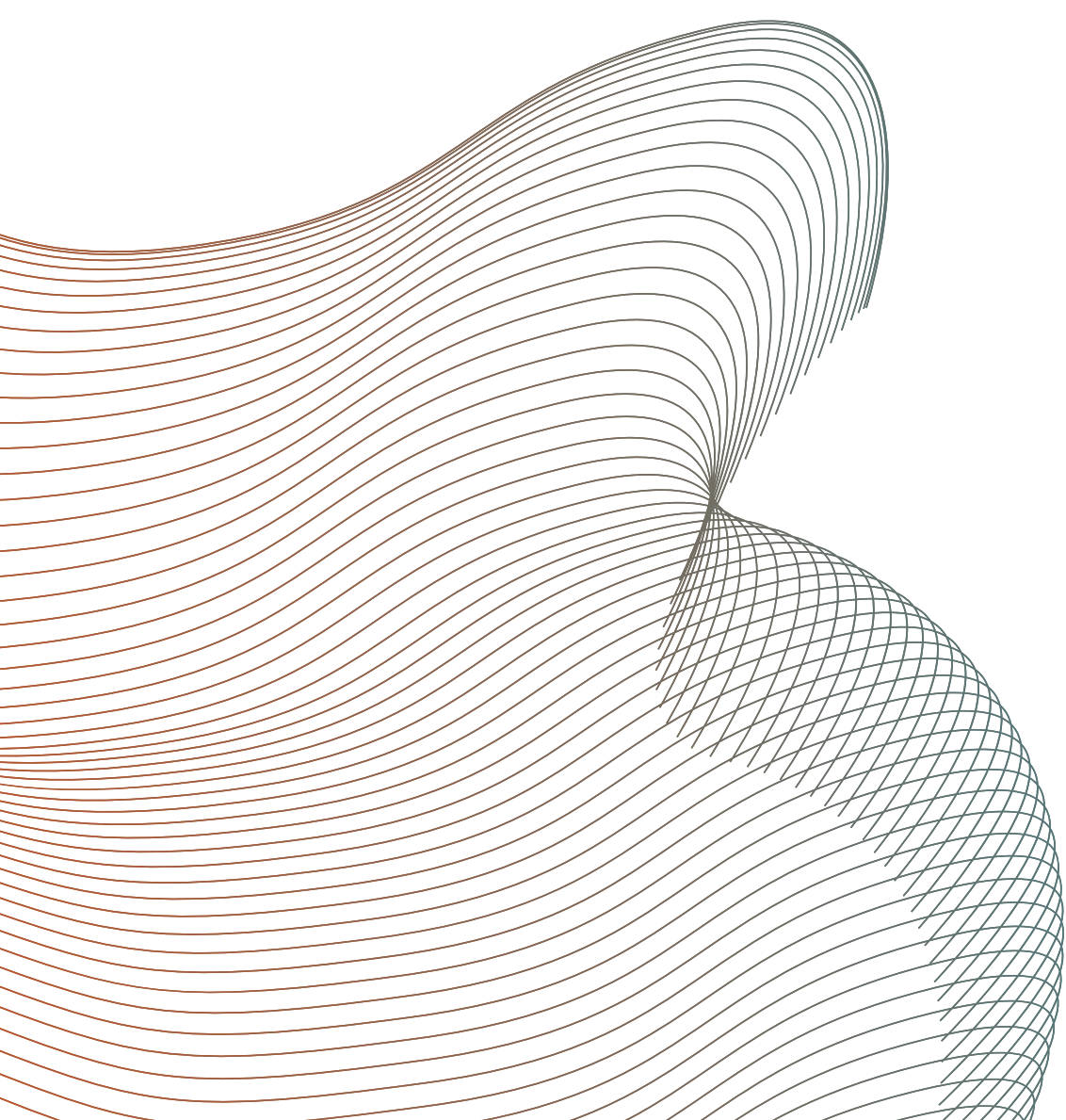
Model	Key Hyperparameters	Precision	Recall	F1-Score
K-Nearest Neighbors	<i>n_neighbors: 2000</i> <i>(Features Scaled)</i>	0.94	0.52	0.67
Random Forest	<i>n_estimators: 100</i> <i>class_weight: 'balanced'</i>	0.81	0.55	0.65
Gradient Boosting	<i>n_estimators: 500, max_depth: 5</i> <i>learning_rate: 0.2</i>	0.80	0.25	0.39

- Gradient Boosting struggled heavily (39%).
- KNN achieved the highest overall score but required strict feature scaling.
- Random Forest handled the imbalance naturally via class weights.

ML Models

Feature	K-Nearest Neighbors (F1: 0.67)	Random Forest (F1: 0.65)
Inference Type	Lazy Learner (Calculates on the fly)	Eager Learner (Pre-computed rules)
Memory Footprint	High: Must store 80k training rows	Low: Compact binary tree structure
Latency	Slow: Increases with data size	Fast: Constant millisecond response
Complexity	High: Requires Scaler artifact sync	Low: No feature scaling required

➡ We chose Random Forest for production. Despite KNN’s slightly higher F1-score, Random Forest is faster, uses less memory, and is more robust. It eliminates the need for feature scaling, significantly reducing technical debt and complexity for our MLOps pipeline.



Q & A